PRACTICAL-4

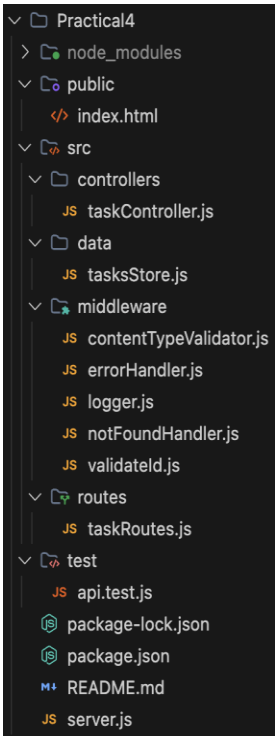
PROBLEM DEFINITION:

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 404, 500) for each response.

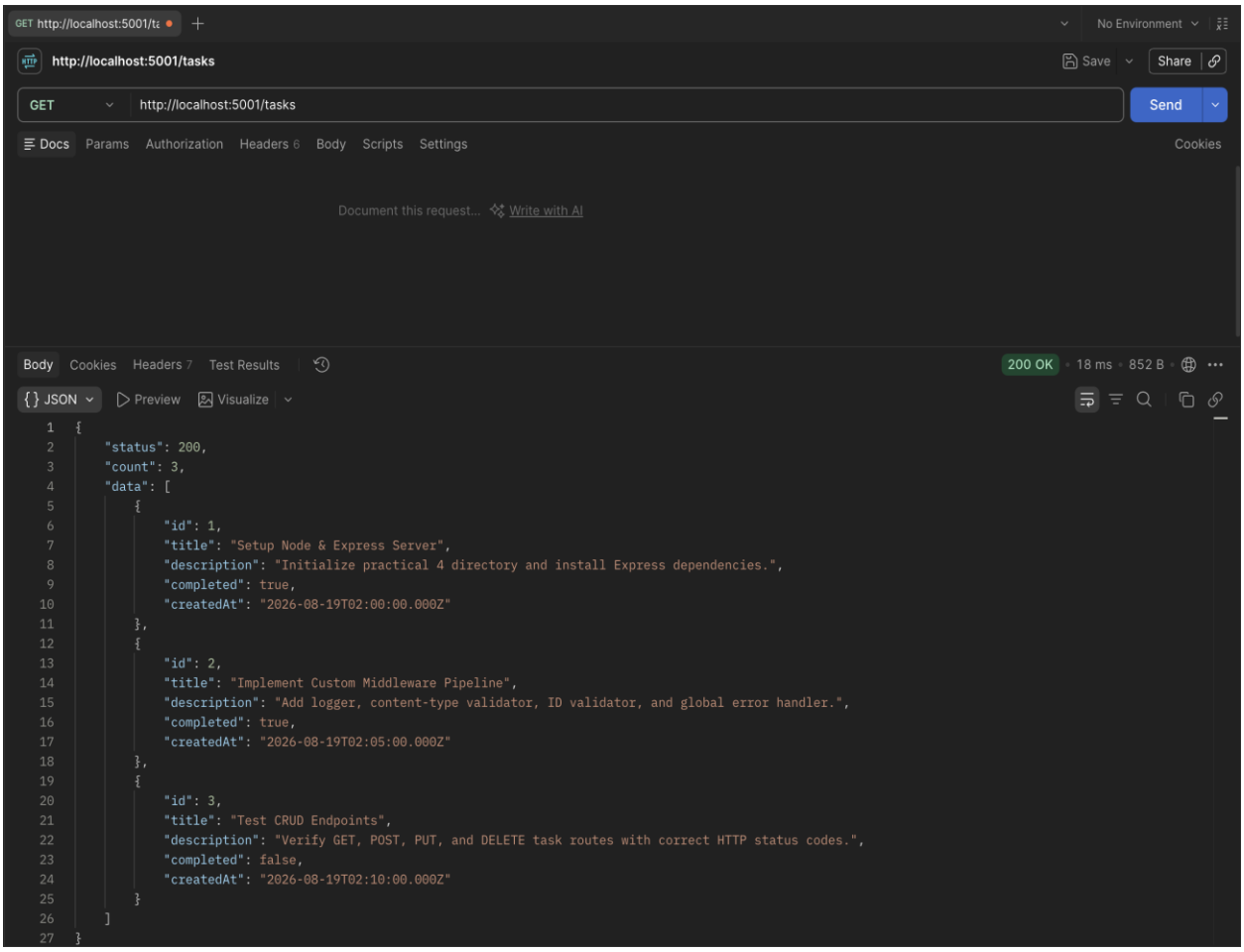
SUPPLEMENTARY PROBLEMS:

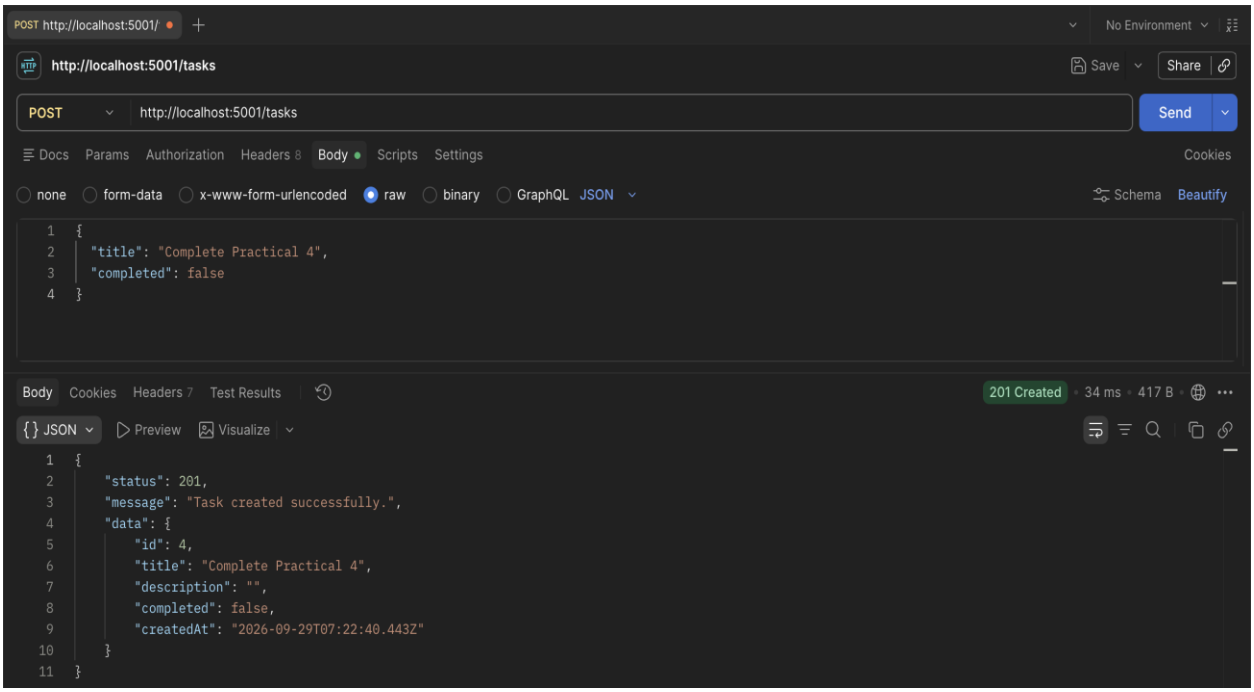
- Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.
- Add a route-specific middleware that validates the task ID format before reaching the controller.
- Add a 404 handler for undefined routes that returns a structured JSON response.

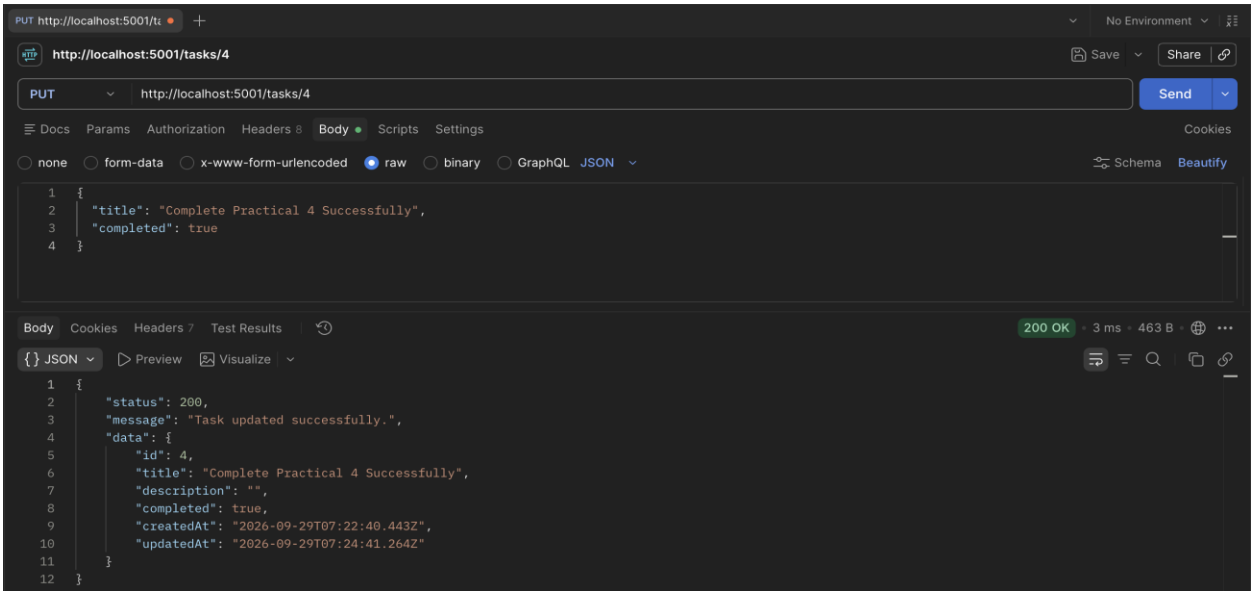
ARCHITECTURE / DIAGRAM:

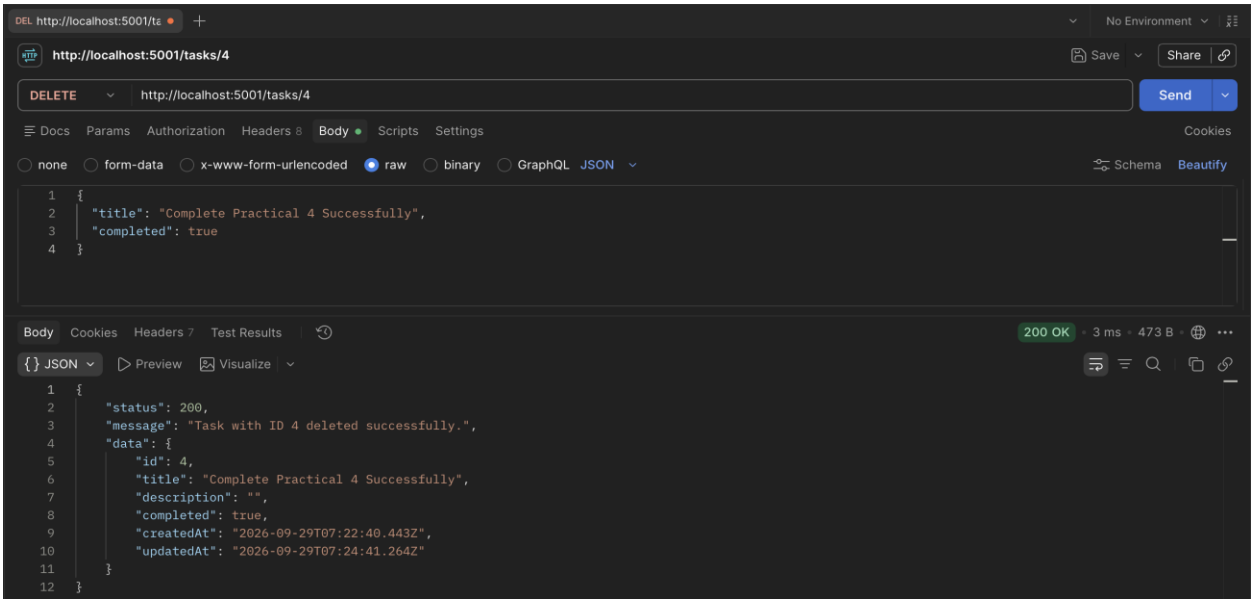


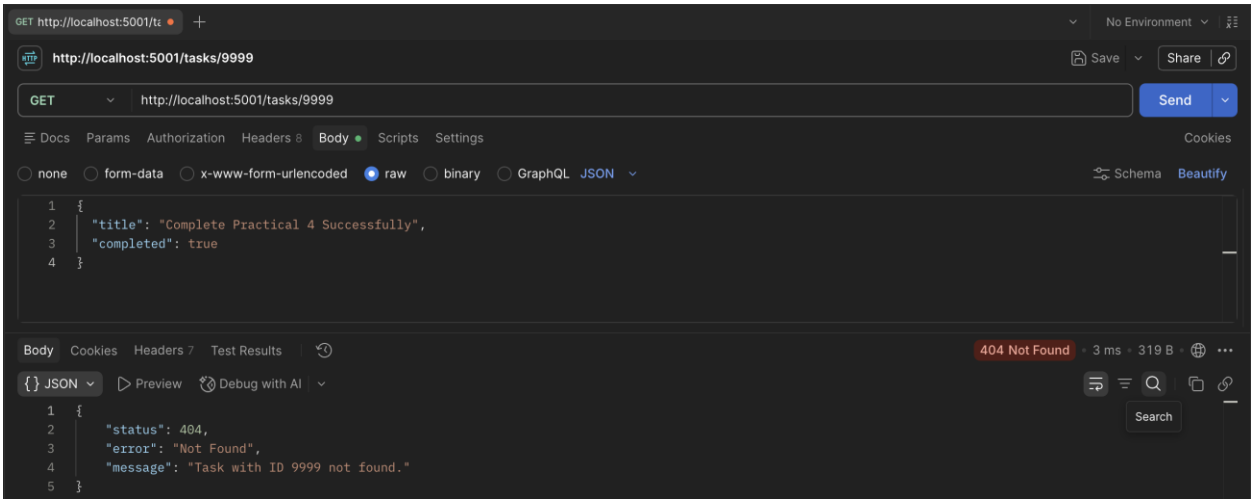
OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

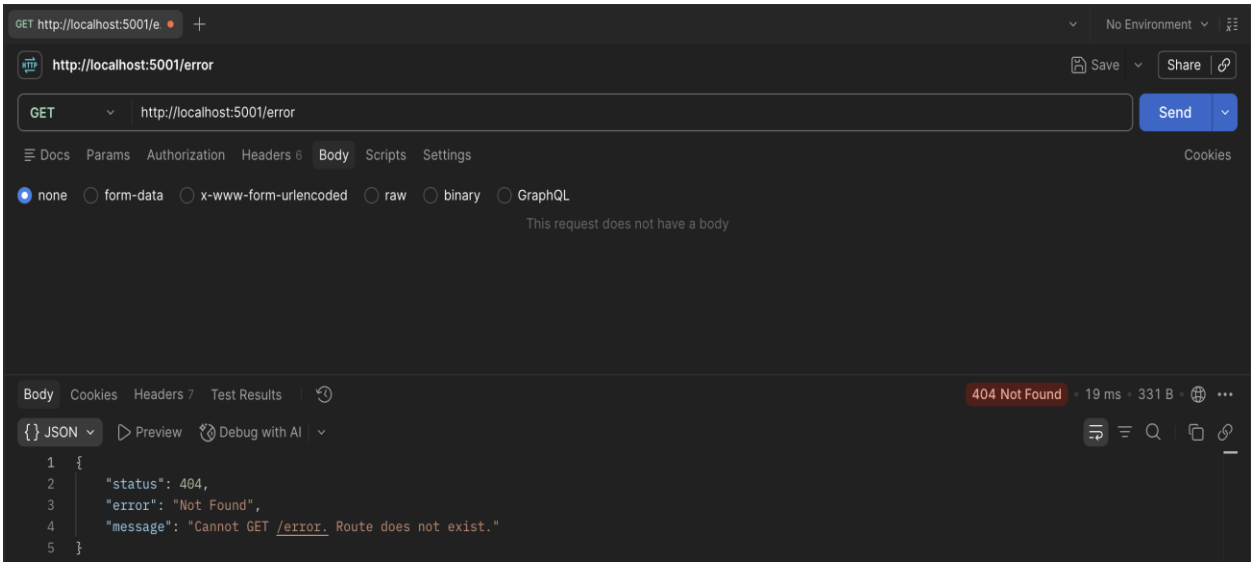


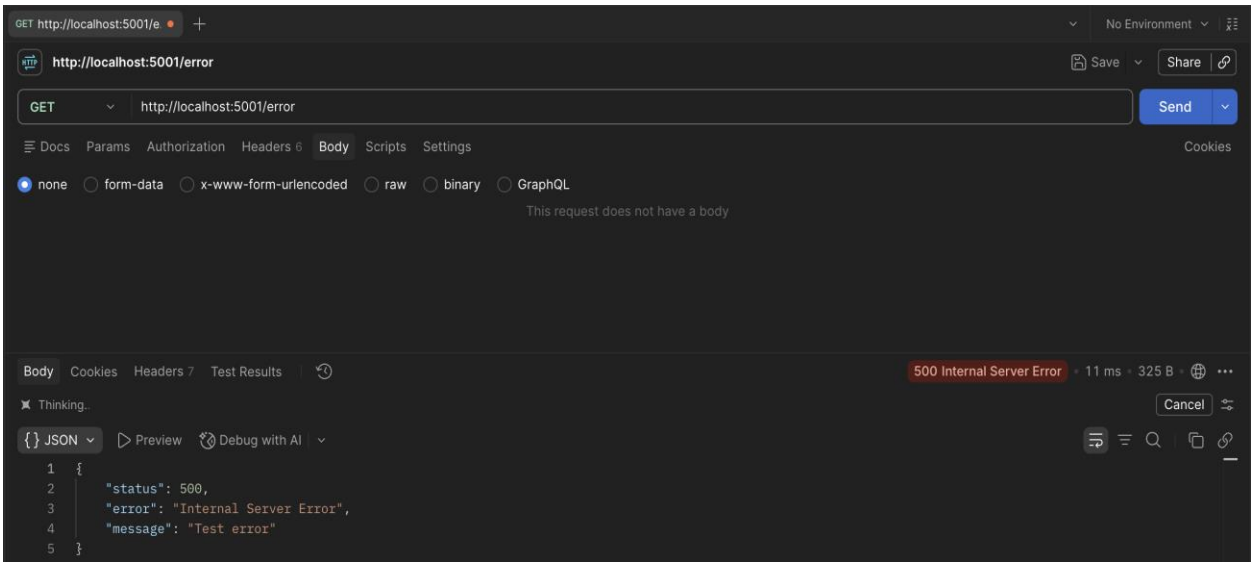


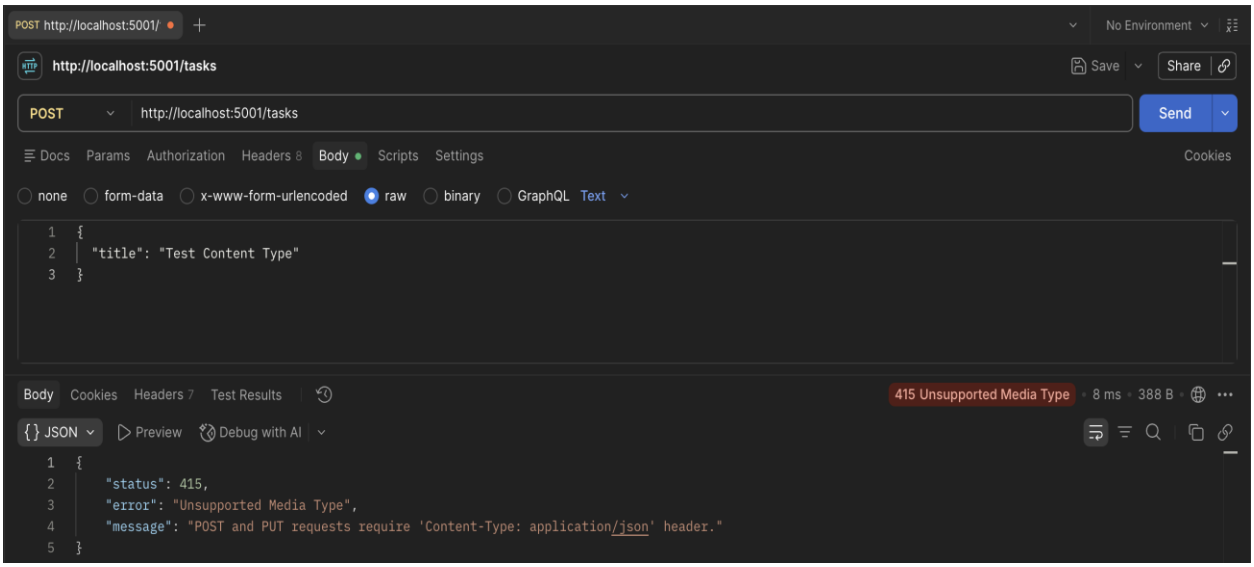

PUT http://localhost:5001/ta
No Environment
http://localhost:5001/tasks/4
Save
Share
PUT
http://localhost:5001/tasks/4
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"title": "Complete Practical 4 Successfully",
"completed": true
}
Body Cookies Headers 7 Test Results
200 OK 3 ms 463 B
{} JSON Preview Visualize
{
"status": 200,
"message": "Task updated successfully.",
"data": {
"id": 4,
"title": "Complete Practical 4 Successfully",
"description": "",
"completed": true,
"createdAt": "2026-09-29T07:22:40.443Z",
"updatedAt": "2026-09-29T07:24:41.264Z"
}
}

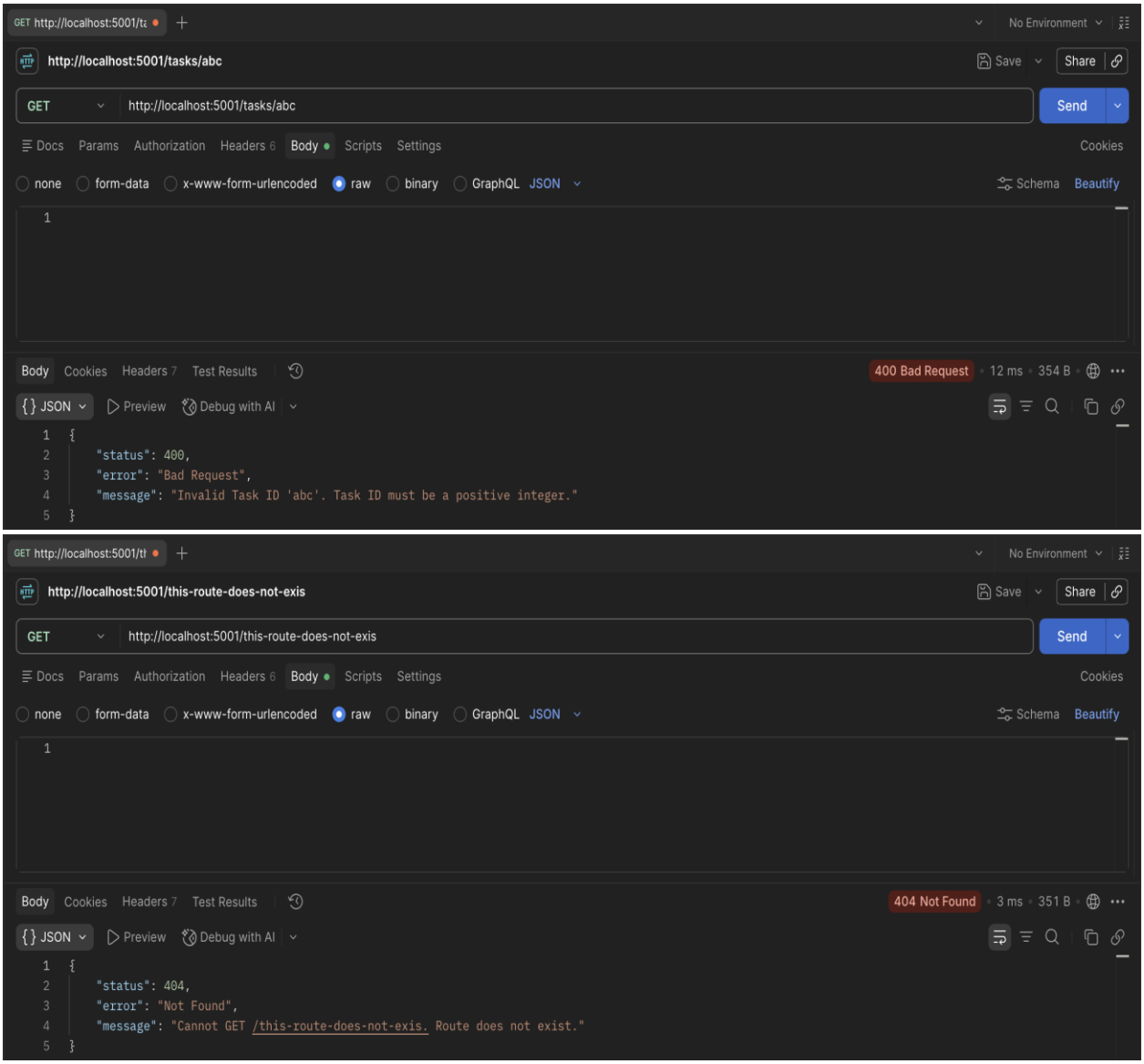

DEL http://localhost:5001/ta
No Environment
http://localhost:5001/tasks/4
Save
Share
DELETE
http://localhost:5001/tasks/4
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"title": "Complete Practical 4 Successfully",
"completed": true
}
Body Cookies Headers 7 Test Results
200 OK 3 ms 473 B
{} JSON Preview Visualize
{
"status": 200,
"message": "Task with ID 4 deleted successfully.",
"data": {
"id": 4,
"title": "Complete Practical 4 Successfully",
"description": "",
"completed": true,
"createdAt": "2026-09-29T07:22:40.443Z",
"updatedAt": "2026-09-29T07:24:41.264Z"
}
}


GET http://localhost:5001/ta
No Environment
http://localhost:5001/tasks/9999
Save
Share
GET
http://localhost:5001/tasks/9999
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"title": "Complete Practical 4 Successfully",
"completed": true
}
Body Cookies Headers 7 Test Results
404 Not Found 3 ms 319 B
{} JSON Preview Debug with AI
Search
{
"status": 404,
"error": "Not Found",
"message": "Task with ID 9999 not found."
}


GET http://localhost:5001/e
No Environment
http://localhost:5001/error
Save
Share
GET
http://localhost:5001/error
Send
Docs Params Authorization Headers 6 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
This request does not have a body
Body Cookies Headers 7 Test Results
404 Not Found 19 ms 331 B
{} JSON Preview Debug with AI
{
"status": 404,
"error": "Not Found",
"message": "Cannot GET /error. Route does not exist."
}


GET http://localhost:5001/e
No Environment
http://localhost:5001/error
Save
Share
GET
http://localhost:5001/error
Send
Docs Params Authorization Headers 6 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
This request does not have a body
Body Cookies Headers 7 Test Results
500 Internal Server Error 11 ms 325 B
Thinking..
Cancel
{} JSON Preview Debug with AI
{
"status": 500,
"error": "Internal Server Error",
"message": "Test error"
}


POST http://localhost:5001/
No Environment
http://localhost:5001/tasks
Save
Share
POST
http://localhost:5001/tasks
Send
Docs Params Authorization Headers 8 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL Text
{
"title": "Test Content Type"
}
Body Cookies Headers 7 Test Results
415 Unsupported Media Type 8 ms 388 B
{} JSON Preview Debug with AI
{
"status": 415,
"error": "Unsupported Media Type",
"message": "POST and PUT requests require 'Content-Type: application/json' header."
}



KEY QUESTIONS / ANALYSIS:

1) Why must the error handling middleware be defined last in the middleware chain?

The error handling middleware should be defined **last** because Express processes middleware in the order it is written. If an error occurs in any previous middleware or route, the error is passed to the error handler using next(err). Placing the error handler at the end ensures that it can catch errors from all the routes and middleware before it.

2) What is the difference between app.use() and route-specific middleware?

- **app.use()** applies middleware globally to all matching requests and routes. For example, a logging middleware using app.use() can log every incoming request.
- **Route-specific middleware** is applied only to a particular route. For example, an ID validation middleware can be used only with PUT /tasks/:id and DELETE /tasks/:id.

Example:

```
app.use(logger); // Global middleware
app.put('/tasks/:id', validateId, updateTask); // Route-specific middleware
```

3) Why is it considered bad practice to send raw error stack traces to the client?

Raw error stack traces may reveal sensitive information about the server, such as file paths, code structure, database details, or internal implementation. This information can help attackers understand the system and find vulnerabilities.

Instead, the server should send a safe and structured error response, such as:

```
{
  "success": false,
  "message": "Internal Server Error"
}
```

This keeps the application secure while allowing detailed error information to remain available in the server logs for debugging.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks